Cryptography

Hash Functions and Applications

Arkady Yerukhimovich

October 5, 2026

Announcements

- No class next week – Fall break
- Midterm 1 on October 19 (in class)
- You may bring two pages of notes (front and back) to the exam
- Project proposals due October 26 – DON'T WAIT

Where Are We Now

- CPA-secure encryption + mode of operation – can encrypt many long messages
- But, CPA security does not protect against chosen-ciphertext attacks
- Solution: add integrity/authentication via a MAC
- Authenticated encryption: encrypt-then-authenticate

Last Time: Encrypt, Then Authenticate

$$c \leftarrow \mathrm{Enc}_{k_E}(m), \qquad t \leftarrow \mathrm{Mac}_{k_M}(c).$$

- Independent keys; authenticate the entire ciphertext, including the IV/nonce.
- Verify before decrypting; reject invalid tags with $\perp$.
- CPA encryption + strong MAC gives CCA confidentiality and ciphertext integrity.

Last Time: Encrypt, Then Authenticate

$$c \leftarrow \mathrm{Enc}_{k_E}(m), \qquad t \leftarrow \mathrm{Mac}_{k_M}(c).$$

- Independent keys; authenticate the entire ciphertext, including the IV/nonce.
- Verify before decrypting; reject invalid tags with $\perp$.
- CPA encryption + strong MAC gives CCA confidentiality and ciphertext integrity.

Just one problem

Last Time: Encrypt, Then Authenticate

$$c \leftarrow \mathrm{Enc}_{k_E}(m), \qquad t \leftarrow \mathrm{Mac}_{k_M}(c).$$

- Independent keys; authenticate the entire ciphertext, including the IV/nonce.
- Verify before decrypting; reject invalid tags with $\perp$.
- CPA encryption + strong MAC gives CCA confidentiality and ciphertext integrity.

Just one problem

Our PRF-based MAC only tags fixed-length inputs. How do we tag a long ciphertext?

Another Way to Authenticate Long Messages

What if we could take a public *digest* of a long message?

$$m = m_1 \| m_2 \| \cdots \| m_\ell \quad \longrightarrow \quad H(m)$$

Then compute

$$t = \mathrm{Mac}_k(H(m)).$$

Another Way to Authenticate Long Messages

What if we could take a public *digest* of a long message?

$$m = m_1 \| m_2 \| \cdots \| m_\ell \quad \longrightarrow \quad H(m)$$

Then compute

$$t = \mathrm{Mac}_k(H(m)).$$

What would we need from H?

Suppose an attacker finds $m \neq m'$ with $H(m) = H(m')$

Another Way to Authenticate Long Messages

What if we could take a public *digest* of a long message?

$$m = m_1 \| m_2 \| \cdots \| m_\ell \quad \longrightarrow \quad H(m)$$

Then compute

$$t = \mathrm{Mac}_k(H(m)).$$

What would we need from H?

Suppose an attacker finds $m \neq m'$ with $H(m) = H(m')$

A digest must fix the message, even though its computation is public.

Outline

1. Hash Functions and Collision Resistance
2. MACs for Long Messages
3. Merkle Trees
4. Other Applications of Hash Functions

Hash Functions

A hash function efficiently maps arbitrary-length inputs to fixed-length digests:

$$H : \{0,1\}^* \longrightarrow \{0,1\}^\ell.$$

Hash Functions

A hash function efficiently maps arbitrary-length inputs to fixed-length digests:

$$H : \{0,1\}^* \longrightarrow \{0,1\}^\ell.$$

- Deterministic: the same input gives the same digest.

Hash Functions

A hash function efficiently maps arbitrary-length inputs to fixed-length digests:

$$H : \{0,1\}^* \longrightarrow \{0,1\}^\ell.$$

- Deterministic: the same input gives the same digest.
- Public: anyone can compute it.

Hash Functions

A hash function efficiently maps arbitrary-length inputs to fixed-length digests:

$$H : \{0,1\}^* \longrightarrow \{0,1\}^\ell.$$

- Deterministic: the same input gives the same digest.
- Public: anyone can compute it.
- Compressing: many different inputs must share a digest.

Hash Functions

A hash function efficiently maps arbitrary-length inputs to fixed-length digests:

$$H : \{0,1\}^* \longrightarrow \{0,1\}^\ell.$$

- Deterministic: the same input gives the same digest.
- Public: anyone can compute it.
- Compressing: many different inputs must share a digest.

The security goal

Hash Functions

A hash function efficiently maps arbitrary-length inputs to fixed-length digests:

$$H : \{0,1\}^* \longrightarrow \{0,1\}^\ell.$$

- Deterministic: the same input gives the same digest.
- Public: anyone can compute it.
- Compressing: many different inputs must share a digest.

The security goal

- Collisions exist!

Hash Functions

A hash function efficiently maps arbitrary-length inputs to fixed-length digests:

$$H : \{0,1\}^* \longrightarrow \{0,1\}^\ell.$$

- Deterministic: the same input gives the same digest.
- Public: anyone can compute it.
- Compressing: many different inputs must share a digest.

The security goal

- Collisions exist!
- It should be computationally hard to find two colliding inputs

Hash Function Security Goals

The hash function H is public in all three settings.

Security goal	Given	Attacker must find

Hash Function Security Goals

The hash function H is public in all three settings.

Security goal	**Given**	**Attacker must find**
Preimage resistance	Target digest y	A message m such that $H(m) = y$

Hash Function Security Goals

The hash function H is public in all three settings.

Security goal	**Given**	**Attacker must find**
Preimage resistance	Target digest y	A message m such that $H(m) = y$
Second-preimage resistance	Message m	A different message $m' \neq m$ with $H(m') = H(m)$

Hash Function Security Goals

The hash function H is public in all three settings.

Security goal	**Given**	**Attacker must find**
Preimage resistance	Target digest y	A message m such that $H(m) = y$
Second-preimage resistance	Message m	A different message $m' \neq m$ with $H(m') = H(m)$
Collision resistance	No fixed target	Any two messages $m \neq m'$ with $H(m) = H(m')$

Hash Function Security Goals

The hash function H is public in all three settings.

Security goal	**Given**	**Attacker must find**
Preimage resistance	Target digest y	A message m such that $H(m) = y$
Second-preimage resistance	Message m	A different message $m' \neq m$ with $H(m') = H(m)$
Collision resistance	No fixed target	Any two messages $m \neq m'$ with $H(m) = H(m')$

Important

In collision resistance, the adversary chooses *both* messages.

Defining Collision Resistance

A family has public seed $s \leftarrow \mathrm{Gen}_H(1^n)$ and functions

$$H^s : \{0,1\}^* \longrightarrow \{0,1\}^{\ell(n)}.$$

Defining Collision Resistance

A family has public seed $s \leftarrow \mathrm{Gen}_H(1^n)$ and functions

$$H^s : \{0,1\}^* \longrightarrow \{0,1\}^{\ell(n)}.$$

$\mathrm{Hash\text{-}Coll}_{\mathcal{A},H}(n)$

Defining Collision Resistance

A family has public seed $s \leftarrow \mathrm{Gen}_H(1^n)$ and functions

$$H^s : \{0,1\}^* \longrightarrow \{0,1\}^{\ell(n)}.$$

$\mathrm{Hash\text{-}Coll}_{\mathcal{A},H}(n)$

1. Challenger samples s and gives it to $\mathcal{A}$.

Defining Collision Resistance

A family has public seed $s \leftarrow \mathrm{Gen}_H(1^n)$ and functions

$$H^s : \{0,1\}^* \longrightarrow \{0,1\}^{\ell(n)}.$$

$\mathrm{Hash\text{-}Coll}_{\mathcal{A},H}(n)$

1. Challenger samples s and gives it to $\mathcal{A}$.
2. $\mathcal{A}$ outputs (m, m').

Defining Collision Resistance

A family has public seed $s \leftarrow \mathrm{Gen}_H(1^n)$ and functions

$$H^s : \{0,1\}^* \longrightarrow \{0,1\}^{\ell(n)}.$$

$\text{Hash-Coll}_{\mathcal{A},H}(n)$

1. Challenger samples s and gives it to $\mathcal{A}$.
2. $\mathcal{A}$ outputs (m, m').
3. $\mathcal{A}$ wins if $m \neq m'$ and $H^s(m) = H^s(m')$.

Defining Collision Resistance

A family has public seed $s \leftarrow \mathrm{Gen}_H(1^n)$ and functions

$$H^s : \{0,1\}^* \longrightarrow \{0,1\}^{\ell(n)}.$$

$\text{Hash-Coll}_{\mathcal{A},H}(n)$

1. Challenger samples s and gives it to $\mathcal{A}$.
2. $\mathcal{A}$ outputs (m, m').
3. $\mathcal{A}$ wins if $m \neq m'$ and $H^s(m) = H^s(m')$.

Collision resistance: for every PPT $\mathcal{A}$,

$$\Pr[\text{Hash-Coll}_{\mathcal{A},H}(n) = 1] \leq \mathrm{negl}(n).$$

A Hash Is Not a PRF or a MAC

Hash-Coll$_{\mathcal{A},H}(n)$

1. Challenger samples s and gives it to $\mathcal{A}$.
2. $\mathcal{A}$ outputs (m, m').
3. $\mathcal{A}$ wins if $m \neq m'$ and $H^s(m) = H^s(m')$.

Comparison to a PRF:

A Hash Is Not a PRF or a MAC

Hash-Coll$_{\mathcal{A},H}(n)$

1. Challenger samples s and gives it to $\mathcal{A}$.
2. $\mathcal{A}$ outputs (m, m').
3. $\mathcal{A}$ wins if $m \neq m'$ and $H^s(m) = H^s(m')$.

Comparison to a PRF:

- The key s is given to $\mathcal{A}$ (in PRF key had to be secret)

A Hash Is Not a PRF or a MAC

Hash-Coll$_{\mathcal{A},H}(n)$

1. Challenger samples s and gives it to $\mathcal{A}$.
2. $\mathcal{A}$ outputs (m, m').
3. $\mathcal{A}$ wins if $m \neq m'$ and $H^s(m) = H^s(m')$.

Comparison to a PRF:

- The key s is given to $\mathcal{A}$ (in PRF key had to be secret)
- No oracle is necessary, $\mathcal{A}$ has s, so can compute $H^s(\cdot)$

A Hash Is Not a PRF or a MAC

Hash-Coll$_{\mathcal{A},H}(n)$

1. Challenger samples s and gives it to $\mathcal{A}$.
2. $\mathcal{A}$ outputs (m, m').
3. $\mathcal{A}$ wins if $m \neq m'$ and $H^s(m) = H^s(m')$.

Comparison to a PRF:

- The key s is given to $\mathcal{A}$ (in PRF key had to be secret)
- No oracle is necessary, $\mathcal{A}$ has s, so can compute $H^s(\cdot)$

Comparison to a MAC:

A Hash Is Not a PRF or a MAC

Hash-Coll$_{\mathcal{A},H}(n)$

1. Challenger samples s and gives it to $\mathcal{A}$.
2. $\mathcal{A}$ outputs (m, m').
3. $\mathcal{A}$ wins if $m \neq m'$ and $H^s(m) = H^s(m')$.

Comparison to a PRF:

- The key s is given to $\mathcal{A}$ (in PRF key had to be secret)
- No oracle is necessary, $\mathcal{A}$ has s, so can compute $H^s(\cdot)$

Comparison to a MAC:

- Given $y = H^s(m)$, hard to find m' that hashes to y

A Hash Is Not a PRF or a MAC

Hash-Coll$_{\mathcal{A},H}(n)$

1. Challenger samples s and gives it to $\mathcal{A}$.
2. $\mathcal{A}$ outputs (m, m').
3. $\mathcal{A}$ wins if $m \neq m'$ and $H^s(m) = H^s(m')$.

Comparison to a PRF:

- The key s is given to $\mathcal{A}$ (in PRF key had to be secret)
- No oracle is necessary, $\mathcal{A}$ has s, so can compute $H^s(\cdot)$

Comparison to a MAC:

- Given $y = H^s(m)$, hard to find m' that hashes to y
- But, since s is public, any party can produce $(m', y' = H^s(m'))$

Hash Functions in Practice

Hash functions used in practice are a bit different:

Hash Functions in Practice

Hash functions used in practice are a bit different:

- Output length is fixed (e.g., 256-bits), not a function of n

Hash Functions in Practice

Hash functions used in practice are a bit different:

- Output length is fixed (e.g., 256-bits), not a function of n
- Usually used unkeyed

Hash Functions in Practice

Hash functions used in practice are a bit different:

- Output length is fixed (e.g., 256-bits), not a function of n
- Usually used unkeyed
- Still generally believed to be collision resistant

Hash Functions in Practice

Hash functions used in practice are a bit different:

- Output length is fixed (e.g., 256-bits), not a function of n
- Usually used unkeyed
- Still generally believed to be collision resistant

For now, we will stick to the asymptotic definition

How Secure Can a Hash Function Be?

A hash function maps arbitrary length strings to $\ell(n)$ length strings:

How Secure Can a Hash Function Be?

A hash function maps arbitrary length strings to $\ell(n)$ length strings:

- There are more possible inputs than outputs, collisions must occur

How Secure Can a Hash Function Be?

A hash function maps arbitrary length strings to $\ell(n)$ length strings:

- There are more possible inputs than outputs, collisions must occur
- If hash cuts input in half: $H : \{0,1\}^{2\ell} \to \{0,1\}^{\ell}$, then $2^{2\ell}$ values mapped to 2^{ℓ} boxes, so there are very many collisions

How Secure Can a Hash Function Be?

A hash function maps arbitrary length strings to $\ell(n)$ length strings:

- There are more possible inputs than outputs, collisions must occur
- If hash cuts input in half: $H : \{0,1\}^{2\ell} \to \{0,1\}^{\ell}$, then $2^{2\ell}$ values mapped to 2^{ℓ} boxes, so there are very many collisions
- How many values do I need to try before I find such a collision?

How Secure Can a Hash Function Be?

A hash function maps arbitrary length strings to $\ell(n)$ length strings:

- There are more possible inputs than outputs, collisions must occur
- If hash cuts input in half: $H : \{0,1\}^{2\ell} \to \{0,1\}^{\ell}$, then $2^{2\ell}$ values mapped to 2^{ℓ} boxes, so there are very many collisions
- How many values do I need to try before I find such a collision?

Birthday Paradox

How Secure Can a Hash Function Be?

A hash function maps arbitrary length strings to $\ell(n)$ length strings:

- There are more possible inputs than outputs, collisions must occur
- If hash cuts input in half: $H : \{0,1\}^{2\ell} \to \{0,1\}^{\ell}$, then $2^{2\ell}$ values mapped to 2^{ℓ} boxes, so there are very many collisions
- How many values do I need to try before I find such a collision?

Birthday Paradox

How many people do you need before you expect two to have same birthday?

How Secure Can a Hash Function Be?

A hash function maps arbitrary length strings to $\ell(n)$ length strings:

- There are more possible inputs than outputs, collisions must occur
- If hash cuts input in half: $H : \{0,1\}^{2\ell} \to \{0,1\}^{\ell}$, then $2^{2\ell}$ values mapped to 2^{ℓ} boxes, so there are very many collisions
- How many values do I need to try before I find such a collision?

Birthday Paradox

How many people do you need before you expect two to have same birthday?

- Tempting to say 365, but this is wrong.

How Secure Can a Hash Function Be?

A hash function maps arbitrary length strings to $\ell(n)$ length strings:

- There are more possible inputs than outputs, collisions must occur
- If hash cuts input in half: $H : \{0,1\}^{2\ell} \to \{0,1\}^{\ell}$, then $2^{2\ell}$ values mapped to 2^{ℓ} boxes, so there are very many collisions
- How many values do I need to try before I find such a collision?

Birthday Paradox

How many people do you need before you expect two to have same birthday?

- Tempting to say 365, but this is wrong.
- Consider all pairs of people (p_i, p_j) for $i \neq j$

How Secure Can a Hash Function Be?

A hash function maps arbitrary length strings to $\ell(n)$ length strings:

- There are more possible inputs than outputs, collisions must occur
- If hash cuts input in half: $H : \{0,1\}^{2\ell} \to \{0,1\}^{\ell}$, then $2^{2\ell}$ values mapped to 2^{ℓ} boxes, so there are very many collisions
- How many values do I need to try before I find such a collision?

Birthday Paradox

How many people do you need before you expect two to have same birthday?

- Tempting to say 365, but this is wrong.
- Consider all pairs of people (p_i, p_j) for $i \neq j$
- There are $\binom{n}{2} = \frac{n(n-1)}{2} = O(n^2)$ such pairs

How Secure Can a Hash Function Be?

A hash function maps arbitrary length strings to $\ell(n)$ length strings:

- There are more possible inputs than outputs, collisions must occur
- If hash cuts input in half: $H : \{0,1\}^{2\ell} \to \{0,1\}^{\ell}$, then $2^{2\ell}$ values mapped to 2^{ℓ} boxes, so there are very many collisions
- How many values do I need to try before I find such a collision?

Birthday Paradox

How many people do you need before you expect two to have same birthday?

- Tempting to say 365, but this is wrong.
- Consider all pairs of people (p_i, p_j) for $i \neq j$
- There are $\binom{n}{2} = \frac{n(n-1)}{2} = O(n^2)$ such pairs
- For a pair, $\Pr[bday_i = bday_j] = 1/365$

How Secure Can a Hash Function Be?

A hash function maps arbitrary length strings to $\ell(n)$ length strings:

- There are more possible inputs than outputs, collisions must occur
- If hash cuts input in half: $H : \{0,1\}^{2\ell} \to \{0,1\}^{\ell}$, then $2^{2\ell}$ values mapped to 2^{ℓ} boxes, so there are very many collisions
- How many values do I need to try before I find such a collision?

Birthday Paradox

How many people do you need before you expect two to have same birthday?

- Tempting to say 365, but this is wrong.
- Consider all pairs of people (p_i, p_j) for $i \neq j$
- There are $\binom{n}{2} = \frac{n(n-1)}{2} = O(n^2)$ such pairs
- For a pair, $\Pr[bday_i = bday_j] = 1/365$
- After 365 pairs (28 people), expect a collision

How Secure Can a Hash Function Be?

A hash function maps arbitrary length strings to $\ell(n)$ length strings:

- There are more possible inputs than outputs, collisions must occur
- If hash cuts input in half: $H : \{0,1\}^{2\ell} \to \{0,1\}^{\ell}$, then $2^{2\ell}$ values mapped to 2^{ℓ} boxes, so there are very many collisions
- How many values do I need to try before I find such a collision?

Birthday Paradox

How many people do you need before you expect two to have same birthday?

- Tempting to say 365, but this is wrong.
- Consider all pairs of people (p_i, p_j) for $i \neq j$
- There are $\binom{n}{2} = \frac{n(n-1)}{2} = O(n^2)$ such pairs
- For a pair, $\Pr[bday_i = bday_j] = 1/365$
- After 365 pairs (28 people), expect a collision
- Generally, $O(2^{\ell/2})$ for output length ℓ – need ℓ large enough

Activity: Choose a Digest Length

Work in a group for 5 minutes.

1. A digest has 80 bits. Roughly how many trials find a collision?
2. You want about 2^{128} generic collision-search work. How many digest bits do you need?
3. For an ideal 256-bit hash, does finding a preimage of a fixed target take 2^{128} work or 2^{256} work?
4. A developer truncates a 256-bit digest to 128 bits. What collision-search work remains?

Explain each answer by identifying whether the attacker chooses one message or both.

Break

10 minutes

Outline

1. Hash Functions and Collision Resistance
2. MACs for Long Messages
3. Merkle Trees
4. Other Applications of Hash Functions

Hash-and-MAC

Our Goal

We need MAC that can authenticate long messages

Hash-and-MAC

Our Goal

We need MAC that can authenticate long messages

Building blocks

- Collision-resistant hash H with ℓ-bit output
- Secure MAC for ℓ-bit messages.

Hash-and-MAC

Our Goal

We need MAC that can authenticate long messages

Building blocks

- Collision-resistant hash H with ℓ-bit output
- Secure MAC for ℓ-bit messages.

Choose independent $k \leftarrow \mathrm{Gen}_M(1^n)$ and $s \leftarrow \mathrm{Gen}_H(1^n)$. Keep k secret; publish s.

Hash-and-MAC

Our Goal

We need MAC that can authenticate long messages

Building blocks

- Collision-resistant hash H with ℓ-bit output
- Secure MAC for ℓ-bit messages.

Choose independent $k \leftarrow \mathrm{Gen}_M(1^n)$ and $s \leftarrow \mathrm{Gen}_H(1^n)$. Keep k secret; publish s.

$$\mathrm{Mac}'_{k,s}(m) = \mathrm{Mac}_k(H^s(m)),$$

Hash-and-MAC

Our Goal

We need MAC that can authenticate long messages

Building blocks

- Collision-resistant hash H with ℓ-bit output
- Secure MAC for ℓ-bit messages.

Choose independent $k \leftarrow \mathrm{Gen}_M(1^n)$ and $s \leftarrow \mathrm{Gen}_H(1^n)$. Keep k secret; publish s.

$$\mathrm{Mac}'_{k,s}(m) = \mathrm{Mac}_k(H^s(m)),$$

$$\mathrm{Verify}'_{k,s}(m,t) = \mathrm{Verify}_k(H^s(m),t).$$

Hash-and-MAC

Our Goal

We need MAC that can authenticate long messages

Building blocks

- Collision-resistant hash H with ℓ-bit output
- Secure MAC for ℓ-bit messages.

Choose independent $k \leftarrow \mathrm{Gen}_M(1^n)$ and $s \leftarrow \mathrm{Gen}_H(1^n)$. Keep k secret; publish s.

$$\mathrm{Mac}'_{k,s}(m) = \mathrm{Mac}_k(H^s(m)),$$

$$\mathrm{Verify}'_{k,s}(m,t) = \mathrm{Verify}_k(H^s(m),t).$$

- Hash the whole message; tag the short digest.
- The receiver recomputes the digest before verifying the tag.

Security of Hash-and-MAC

Hash-and-MAC

Construct $\Pi' = (\mathsf{Gen}', \mathsf{Mac}', \mathsf{Verify}')$:

- $\mathsf{Gen}'(1^n)$: Choose $k \leftarrow \mathsf{Gen}(1^n)$, $s \leftarrow \mathsf{Gen}_H(1^n)$. Output $k' = (k, s)$
- $\mathsf{Mac}'_{k'}(m)$: Recall that $k' = (k, s)$, Output $t = \mathsf{Mac}_k(H^s(m))$
- $\mathsf{Verify}'_{k'}(m, t)$: Output 1 if $\mathsf{Verify}_k(H^s(m), t) = 1$

Security of Hash-and-MAC

Hash-and-MAC

Construct $\Pi' = (\mathsf{Gen}', \mathsf{Mac}', \mathsf{Verify}')$:

- $\mathsf{Gen}'(1^n)$: Choose $k \leftarrow \mathsf{Gen}(1^n)$, $s \leftarrow \mathsf{Gen}_H(1^n)$. Output $k' = (k, s)$
- $\mathsf{Mac}'_{k'}(m)$: Recall that $k' = (k, s)$, Output $t = \mathsf{Mac}_k(H^s(m))$
- $\mathsf{Verify}'_{k'}(m, t)$: Output 1 if $\mathsf{Verify}_k(H^s(m), t) = 1$

Proof Sketch: Suppose $\mathcal{A}$ forges a valid t on $m^* \notin Q$

Security of Hash-and-MAC

Hash-and-MAC

Construct $\Pi' = (\text{Gen}', \text{Mac}', \text{Verify}')$:

- $\text{Gen}'(1^n)$: Choose $k \leftarrow \text{Gen}(1^n)$, $s \leftarrow \text{Gen}_H(1^n)$. Output $k' = (k, s)$
- $\text{Mac}'_{k'}(m)$: Recall that $k' = (k, s)$, Output $t = \text{Mac}_k(H^s(m))$
- $\text{Verify}'_{k'}(m, t)$: Output 1 if $\text{Verify}_k(H^s(m), t) = 1$

Proof Sketch: Suppose $\mathcal{A}$ forges a valid t on $m^* \notin Q$

- Case 1: $\exists m \in Q$ s.t. $H^s(m^*) = H^s(m)$

Security of Hash-and-MAC

Hash-and-MAC

Construct $\Pi' = (\text{Gen}', \text{Mac}', \text{Verify}')$:

- $\text{Gen}'(1^n)$: Choose $k \leftarrow \text{Gen}(1^n)$, $s \leftarrow \text{Gen}_H(1^n)$. Output $k' = (k, s)$
- $\text{Mac}'_{k'}(m)$: Recall that $k' = (k, s)$, Output $t = \text{Mac}_k(H^s(m))$
- $\text{Verify}'_{k'}(m, t)$: Output 1 if $\text{Verify}_k(H^s(m), t) = 1$

Proof Sketch: Suppose $\mathcal{A}$ forges a valid t on $m^* \notin Q$

- Case 1: $\exists m \in Q$ s.t. $H^s(m^*) = H^s(m)$
  - Then (m, m^*) is a collision in H

Security of Hash-and-MAC

Hash-and-MAC

Construct $\Pi' = (\text{Gen}', \text{Mac}', \text{Verify}')$:

- $\text{Gen}'(1^n)$: Choose $k \leftarrow \text{Gen}(1^n)$, $s \leftarrow \text{Gen}_H(1^n)$. Output $k' = (k, s)$
- $\text{Mac}'_{k'}(m)$: Recall that $k' = (k, s)$, Output $t = \text{Mac}_k(H^s(m))$
- $\text{Verify}'_{k'}(m, t)$: Output 1 if $\text{Verify}_k(H^s(m), t) = 1$

Proof Sketch: Suppose $\mathcal{A}$ forges a valid t on $m^* \notin Q$

- Case 1: $\exists m \in Q$ s.t. $H^s(m^*) = H^s(m)$
  - Then (m, m^*) is a collision in H
- Case 2: $\forall m \in Q$, $H^s(m^*) \neq H^s(m)$

Security of Hash-and-MAC

Hash-and-MAC

Construct $\Pi' = (\text{Gen}', \text{Mac}', \text{Verify}')$:

- $\text{Gen}'(1^n)$: Choose $k \leftarrow \text{Gen}(1^n)$, $s \leftarrow \text{Gen}_H(1^n)$. Output $k' = (k, s)$
- $\text{Mac}'_{k'}(m)$: Recall that $k' = (k, s)$, Output $t = \text{Mac}_k(H^s(m))$
- $\text{Verify}'_{k'}(m, t)$: Output 1 if $\text{Verify}_k(H^s(m), t) = 1$

Proof Sketch: Suppose $\mathcal{A}$ forges a valid t on $m^* \notin Q$

- Case 1: $\exists m \in Q$ s.t. $H^s(m^*) = H^s(m)$
  - Then (m, m^*) is a collision in H
- Case 2: $\forall m \in Q$, $H^s(m^*) \neq H^s(m)$
  - Let $H^s(Q) = \{H^s(m) | m \in Q\}$

Security of Hash-and-MAC

Hash-and-MAC

Construct $\Pi' = (\text{Gen}', \text{Mac}', \text{Verify}')$:

- $\text{Gen}'(1^n)$: Choose $k \leftarrow \text{Gen}(1^n)$, $s \leftarrow \text{Gen}_H(1^n)$. Output $k' = (k, s)$
- $\text{Mac}'_{k'}(m)$: Recall that $k' = (k, s)$, Output $t = \text{Mac}_k(H^s(m))$
- $\text{Verify}'_{k'}(m, t)$: Output 1 if $\text{Verify}_k(H^s(m), t) = 1$

Proof Sketch: Suppose $\mathcal{A}$ forges a valid t on $m^* \notin Q$

- Case 1: $\exists m \in Q$ s.t. $H^s(m^*) = H^s(m)$
  - Then (m, m^*) is a collision in H
- Case 2: $\forall m \in Q$, $H^s(m^*) \neq H^s(m)$
  - Let $H^s(Q) = \{H^s(m) | m \in Q\}$
  - Then, $H^s(m^*) \notin H^s(Q)$

Security of Hash-and-MAC

Hash-and-MAC

Construct $\Pi' = (\text{Gen}', \text{Mac}', \text{Verify}')$:

- $\text{Gen}'(1^n)$: Choose $k \leftarrow \text{Gen}(1^n)$, $s \leftarrow \text{Gen}_H(1^n)$. Output $k' = (k, s)$
- $\text{Mac}'_{k'}(m)$: Recall that $k' = (k, s)$, Output $t = \text{Mac}_k(H^s(m))$
- $\text{Verify}'_{k'}(m, t)$: Output 1 if $\text{Verify}_k(H^s(m), t) = 1$

Proof Sketch: Suppose $\mathcal{A}$ forges a valid t on $m^* \notin Q$

- Case 1: $\exists m \in Q$ s.t. $H^s(m^*) = H^s(m)$
  - Then (m, m^*) is a collision in H
- Case 2: $\forall m \in Q$, $H^s(m^*) \neq H^s(m)$
  - Let $H^s(Q) = \{H^s(m) | m \in Q\}$
  - Then, $H^s(m^*) \notin H^s(Q)$
  - But, then $\mathcal{A}$ has forged valid tag on new message $H^s(m^*)$

Going Back to Authenticated Encryption

For the fixed-length PRF MAC, we can use

$$t = F_{k_M}(H^s(c)).$$

Going Back to Authenticated Encryption

For the fixed-length PRF MAC, we can use

$$t = F_{k_M}(H^s(c)).$$

- Match the digest length to the PRF input length.

Going Back to Authenticated Encryption

For the fixed-length PRF MAC, we can use

$$t = F_{k_M}(H^s(c)).$$

- Match the digest length to the PRF input length.
- This is deterministic with a unique valid tag per ciphertext; unforgeability therefore gives strong unforgeability.

Going Back to Authenticated Encryption

For the fixed-length PRF MAC, we can use

$$t = F_{k_M}(H^s(c)).$$

- Match the digest length to the PRF input length.
- This is deterministic with a unique valid tag per ciphertext; unforgeability therefore gives strong unforgeability.
- It supplies the long-message MAC assumed in last lecture's encrypt-then-authenticate theorem.

Going Back to Authenticated Encryption

For the fixed-length PRF MAC, we can use

$$t = F_{k_M}(H^s(c)).$$

- Match the digest length to the PRF input length.
- This is deterministic with a unique valid tag per ciphertext; unforgeability therefore gives strong unforgeability.
- It supplies the long-message MAC assumed in last lecture's encrypt-then-authenticate theorem.

Receive (c, t)

Hash c, verify the tag, and only then decrypt c. The authenticated input includes the IV/nonce and all ciphertext blocks.

Outline

1. Hash Functions and Collision Resistance
2. MACs for Long Messages
3. Merkle Trees
4. Other Applications of Hash Functions

Outsourced Storage

The Goal

- Client wants to store files $x_1, \ldots, x_n$ on server
- When he later retrieves file, wants to be sure it wasn't modified

Outsourced Storage

The Goal

- Client wants to store files $x_1, \ldots, x_n$ on server
- When he later retrieves file, wants to be sure it wasn't modified

Try 1: Client stores $h_1 = H(x_1), h_2 = H(x_2), \ldots, h_n = H(x_n)$

Outsourced Storage

The Goal

- Client wants to store files $x_1, \ldots, x_n$ on server
- When he later retrieves file, wants to be sure it wasn't modified

Try 1: Client stores $h_1 = H(x_1), h_2 = H(x_2), \ldots, h_n = H(x_n)$

- Pro: Can quickly verify each file, by recomputing $H(x_i)$

Outsourced Storage

The Goal

- Client wants to store files $x_1, \ldots, x_n$ on server
- When he later retrieves file, wants to be sure it wasn't modified

Try 1: Client stores $h_1 = H(x_1), h_2 = H(x_2), \ldots, h_n = H(x_n)$

- Pro: Can quickly verify each file, by recomputing $H(x_i)$
- Con: Client has to store n items

Outsourced Storage

The Goal

- Client wants to store files $x_1, \ldots, x_n$ on server
- When he later retrieves file, wants to be sure it wasn't modified

Try 1: Client stores $h_1 = H(x_1), h_2 = H(x_2), \ldots, h_n = H(x_n)$

- Pro: Can quickly verify each file, by recomputing $H(x_i)$
- Con: Client has to store n items

Try 2: Client stores $h = H(x_1||x_2|| \cdots ||x_n)$

Outsourced Storage

The Goal

- Client wants to store files $x_1, \ldots, x_n$ on server
- When he later retrieves file, wants to be sure it wasn't modified

Try 1: Client stores $h_1 = H(x_1), h_2 = H(x_2), \ldots, h_n = H(x_n)$

- Pro: Can quickly verify each file, by recomputing $H(x_i)$
- Con: Client has to store n items

Try 2: Client stores $h = H(x_1||x_2|| \cdots ||x_n)$

- Pro: Only ℓ bits of storage needed on client

Outsourced Storage

The Goal

- Client wants to store files $x_1, \ldots, x_n$ on server
- When he later retrieves file, wants to be sure it wasn't modified

Try 1: Client stores $h_1 = H(x_1), h_2 = H(x_2), \ldots, h_n = H(x_n)$

- Pro: Can quickly verify each file, by recomputing $H(x_i)$
- Con: Client has to store n items

Try 2: Client stores $h = H(x_1||x_2|| \cdots ||x_n)$

- Pro: Only ℓ bits of storage needed on client
- Con: Must download all files to verify even a single one

Outsourced Storage

The Goal

- Client wants to store files $x_1, \ldots, x_n$ on server
- When he later retrieves file, wants to be sure it wasn't modified

Try 1: Client stores $h_1 = H(x_1), h_2 = H(x_2), \ldots, h_n = H(x_n)$

- Pro: Can quickly verify each file, by recomputing $H(x_i)$
- Con: Client has to store n items

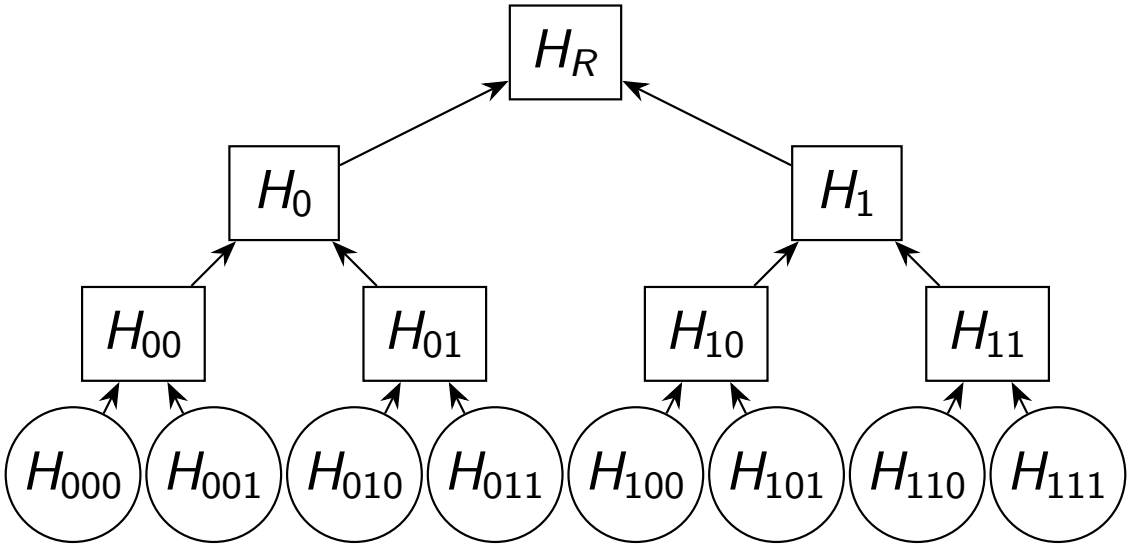
Try 2: Client stores $h = H(x_1||x_2|| \cdots ||x_n)$

- Pro: Only ℓ bits of storage needed on client
- Con: Must download all files to verify even a single one

Question: Can we get solution that achieves both?

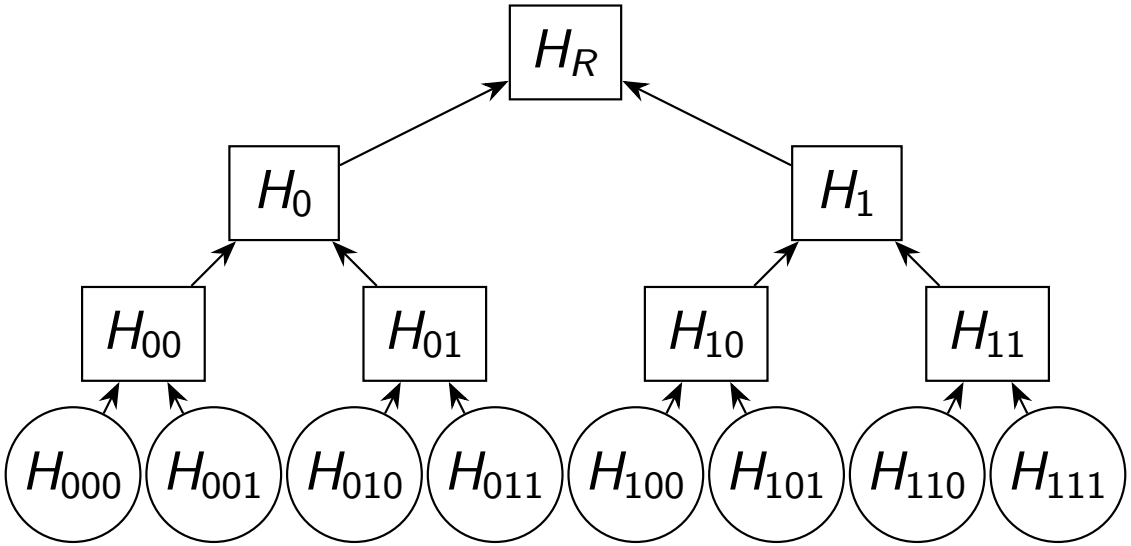
- Low storage on the client
- Low communication to verify a file is correct

A Merkle Tree



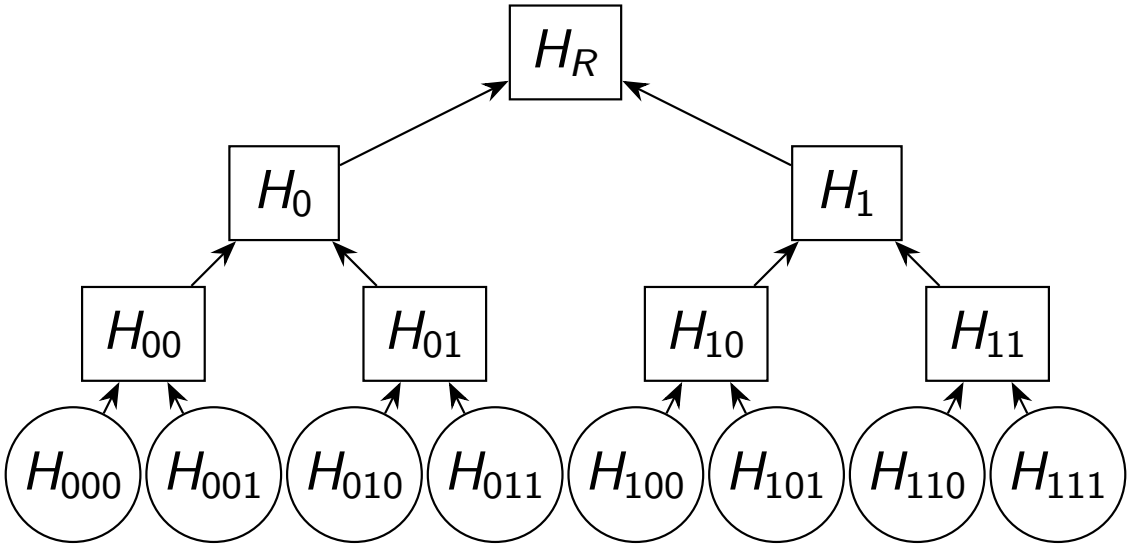
- C has files $x_{000}, \ldots, x_{111}$, computes $H_{000} = H(0 \| x_{000}), \ldots$

A Merkle Tree



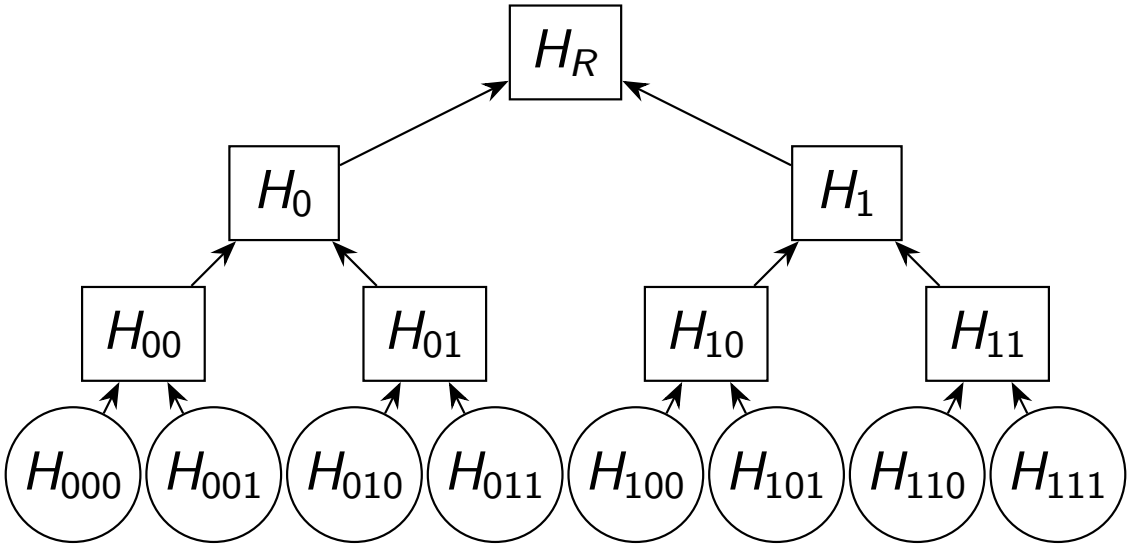
- C has files $x_{000}, \ldots, x_{111}$, computes $H_{000} = H(0 \| x_{000}), \ldots$
- C computes $H_{00} = H(1 \| H_{000} \| H_{001}), H_{01} = H(1 \| H_{010} \| H_{011}), \ldots$

A Merkle Tree



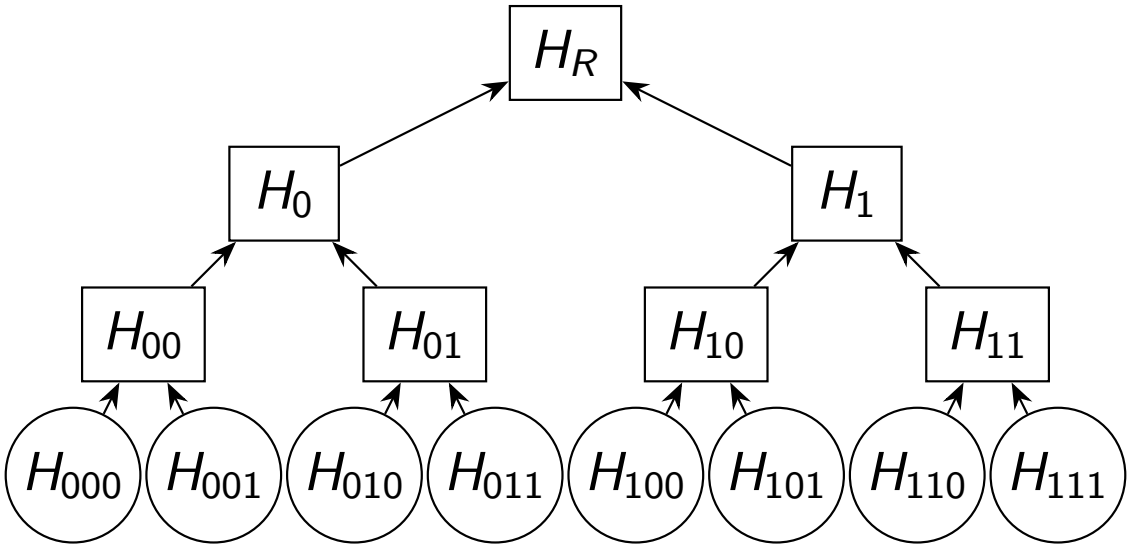
- C has files $x_{000}, \ldots, x_{111}$, computes $H_{000} = H(0 \| x_{000}), \ldots$
- C computes $H_{00} = H(1 \| H_{000} \| H_{001}), H_{01} = H(1 \| H_{010} \| H_{011}), \ldots$
- C does the same for all levels: $H_0 = H(1 \| H_{00} \| H_{01})$, $H_R = H(1 \| H_0 \| H_1)$

A Merkle Tree



- C has files $x_{000}, \ldots, x_{111}$, computes $H_{000} = H(0 \| x_{000}), \ldots$
- C computes $H_{00} = H(1 \| H_{000} \| H_{001}), H_{01} = H(1 \| H_{010} \| H_{011}), \ldots$
- C does the same for all levels: $H_0 = H(1 \| H_{00} \| H_{01})$, $H_R = H(1 \| H_0 \| H_1)$
- C stores value H_R at the root and uploads all files to S

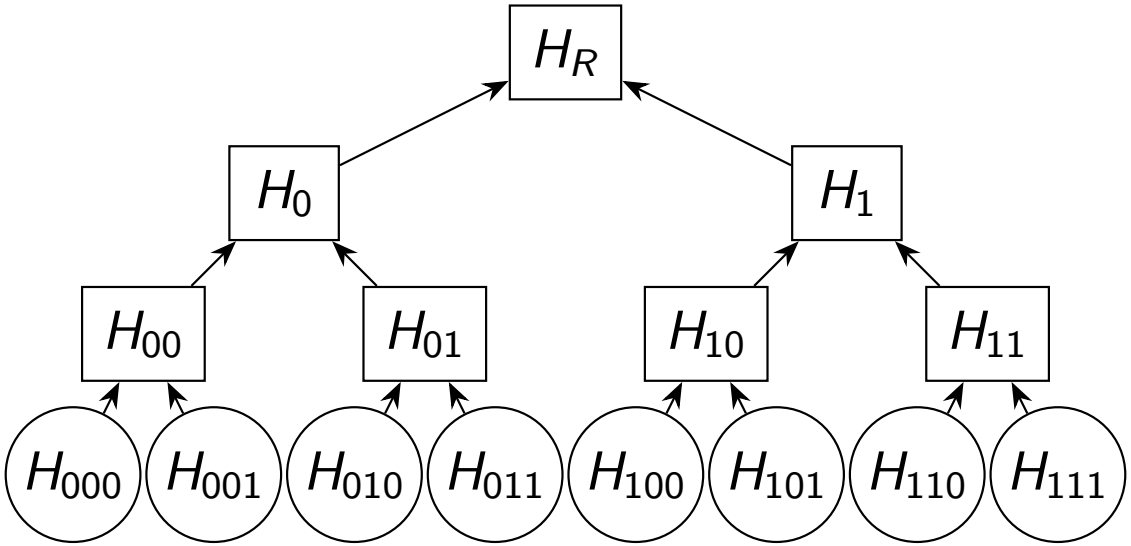
Activity: Verify File x_{001}



1. You know H_R and receive x_{001}. Which additional hashes do you need from the server?
2. Write the three parent computations and the final check.
3. For 2^{20} files, how many sibling digests are needed?

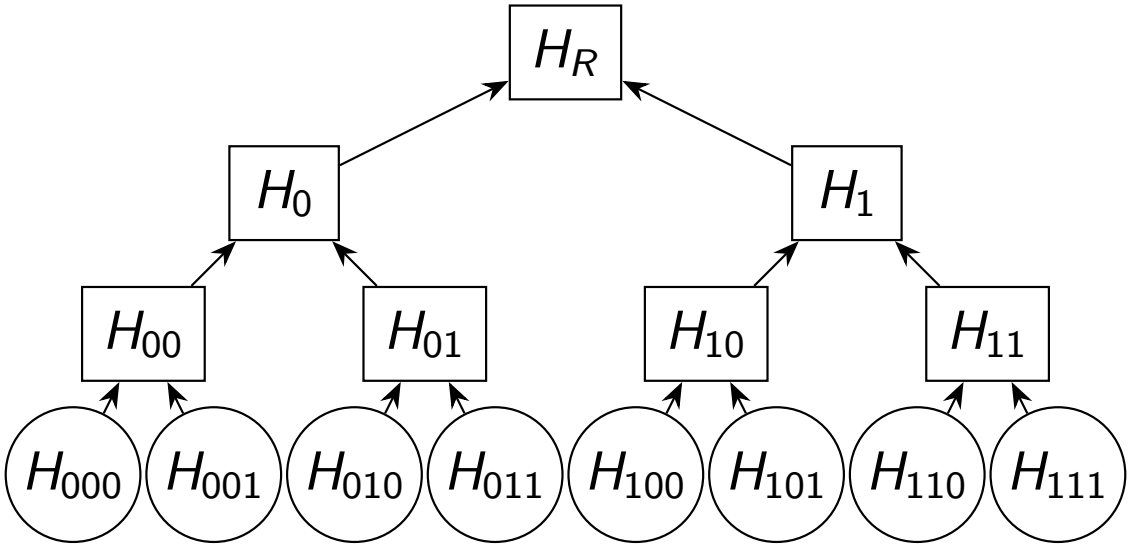
Work in groups for 5 minutes. Mark the path and its siblings.

A Merkle Tree



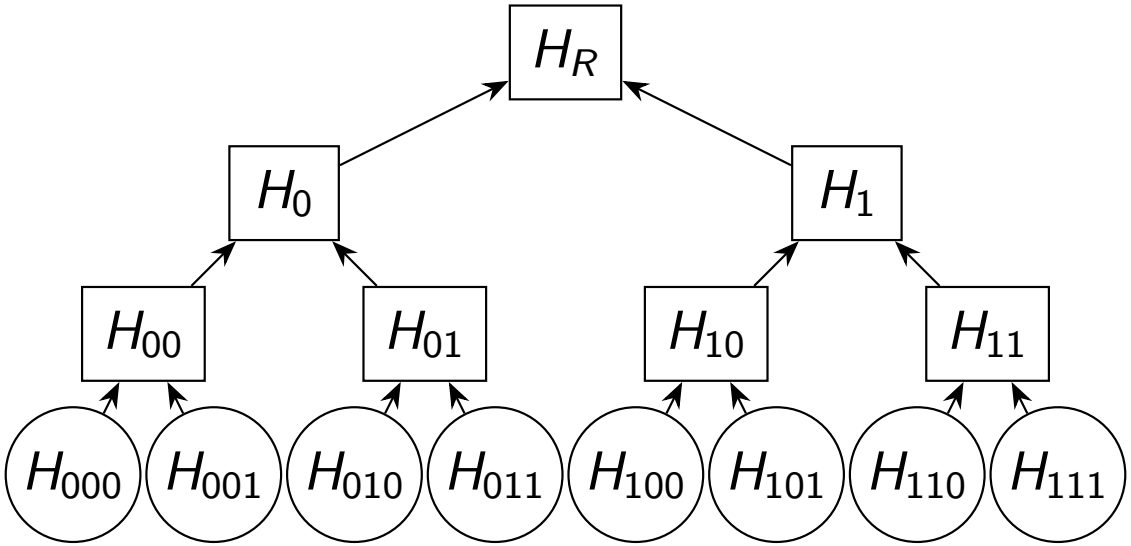
- C has H_R, downloads x_{001} and wants to check it's correct

A Merkle Tree



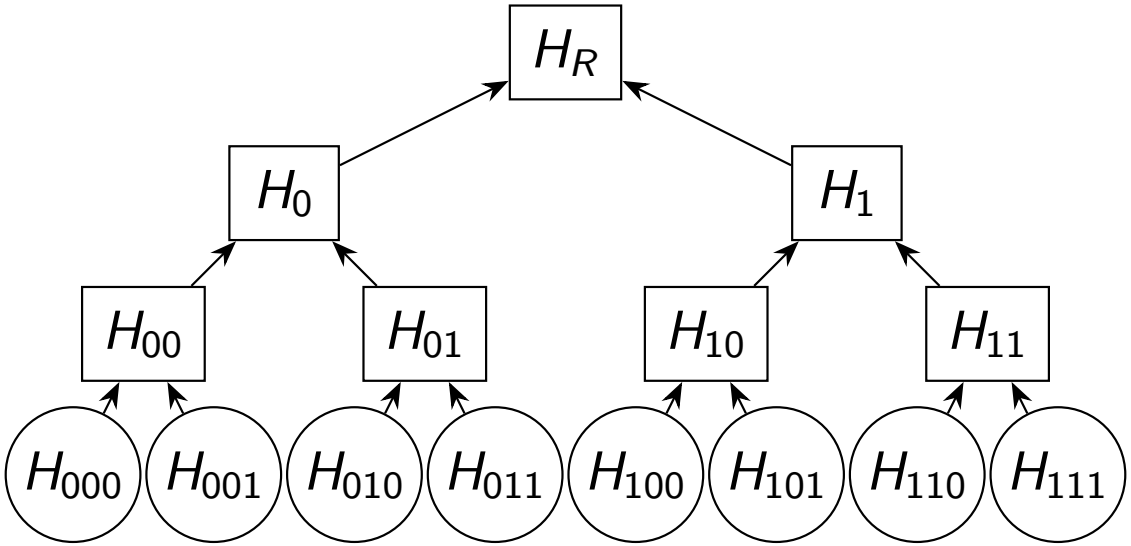
- C has H_R, downloads x_{001} and wants to check it's correct
- S sends C all sibling hashes along the path to root: H_{000}, H_{01}, H_1

A Merkle Tree



- C has H_R, downloads x_{001} and wants to check it's correct
- S sends C all sibling hashes along the path to root: H_{000}, H_{01}, H_1
- C computes H_{001} and can now recompute up to H_R (has both inputs for every node in path)

A Merkle Tree



- C has H_R, downloads x_{001} and wants to check it's correct
- S sends C all sibling hashes along the path to root: H_{000}, H_{01}, H_1
- C computes H_{001} and can now recompute up to H_R (has both inputs for every node in path)
- C checks that computed value is equal to H_R, accepts if so

Security of Merkle Proofs

Suppose a different file x'_{001} is returned in place of x_{001}

Security of Merkle Proofs

Suppose a different file x'_{001} is returned in place of x_{001}

- Compare the honest computation with the forged computation.

Security of Merkle Proofs

Suppose a different file x'_{001} is returned in place of x_{001}

- Compare the honest computation with the forged computation.
- If the changed leaf has the same hash, we already have a collision.

Security of Merkle Proofs

Suppose a different file x'_{001} is returned in place of x_{001}

- Compare the honest computation with the forged computation.
- If the changed leaf has the same hash, we already have a collision.
- Otherwise, follow the different digest upward. The root H_R is the same, so somewhere different encoded inputs must yield the same parent digest.

Merkle Tree Costs and Limits

For a balanced tree with $N = 2^h$ files and ℓ-bit digests:

Merkle Tree Costs and Limits

For a balanced tree with $N = 2^h$ files and ℓ-bit digests:

- Client stores one ℓ-bit root, plus trusted tree metadata.

Merkle Tree Costs and Limits

For a balanced tree with $N = 2^h$ files and ℓ-bit digests:

- Client stores one ℓ-bit root, plus trusted tree metadata.
- Proof contains $h = \log_2 N$ sibling digests.

Merkle Tree Costs and Limits

For a balanced tree with $N = 2^h$ files and ℓ-bit digests:

- Client stores one ℓ-bit root, plus trusted tree metadata.
- Proof contains $h = \log_2 N$ sibling digests.
- Verification hashes the file and computes h parent hashes.

Merkle Tree Costs and Limits

For a balanced tree with $N = 2^h$ files and ℓ-bit digests:

- Client stores one ℓ-bit root, plus trusted tree metadata.
- Proof contains $h = \log_2 N$ sibling digests.
- Verification hashes the file and computes h parent hashes.
- Updating a file changes only its leaf and the path to the root.

Merkle Tree Costs and Limits

For a balanced tree with $N = 2^h$ files and ℓ-bit digests:

- Client stores one ℓ-bit root, plus trusted tree metadata.
- Proof contains $h = \log_2 N$ sibling digests.
- Verification hashes the file and computes h parent hashes.
- Updating a file changes only its leaf and the path to the root.

For $N = 2^{20}$ and $\ell = 256$: 20 sibling digests, or 640 bytes, excluding the file and metadata.

Outline

1. Hash Functions and Collision Resistance
2. MACs for Long Messages
3. Merkle Trees
4. Other Applications of Hash Functions

What Are Hash Functions Good For?

Properties of Hash Functions

- Produce short digest of long strings (e.g., files, etc.)
- Collision resistant – hard to find two strings that have same hash
- $H(x)$ uniquely identifies an item x – hard to find another item x' with same hash

What Are Hash Functions Good For?

Properties of Hash Functions

- Produce short digest of long strings (e.g., files, etc.)
- Collision resistant – hard to find two strings that have same hash
- $H(x)$ uniquely identifies an item x – hard to find another item x' with same hash

Applications of unique identifiers:

What Are Hash Functions Good For?

Properties of Hash Functions

- Produce short digest of long strings (e.g., files, etc.)
- Collision resistant – hard to find two strings that have same hash
- $H(x)$ uniquely identifies an item x – hard to find another item x' with same hash

Applications of unique identifiers:

- Fingerprinting – store $H(x)$ where x is a virus

What Are Hash Functions Good For?

Properties of Hash Functions

- Produce short digest of long strings (e.g., files, etc.)
- Collision resistant – hard to find two strings that have same hash
- $H(x)$ uniquely identifies an item x – hard to find another item x' with same hash

Applications of unique identifiers:

- Fingerprinting – store $H(x)$ where x is a virus
- Deduplication – check if two files (e.g., in cloud storage) are the same

What Are Hash Functions Good For?

Properties of Hash Functions

- Produce short digest of long strings (e.g., files, etc.)
- Collision resistant – hard to find two strings that have same hash
- $H(x)$ uniquely identifies an item x – hard to find another item x' with same hash

Applications of unique identifiers:

- Fingerprinting – store $H(x)$ where x is a virus
- Deduplication – check if two files (e.g., in cloud storage) are the same
- Peer-to-peer file sharing – Use $H(\mathit{file})$ as unique identifier

Password-Based Authentication

How to use a password:

1. User creates password *pwd* when registering for site

Password-Based Authentication

How to use a password:

1. User creates password *pwd* when registering for site
2. Site stores (ID, pwd) on server

Password-Based Authentication

How to use a password:

1. User creates password *pwd* when registering for site
2. Site stores (ID, pwd) on server
3. When logging in, user enters pwd', server compares it to *pwd*

Password-Based Authentication

How to use a password:

1. User creates password *pwd* when registering for site
2. Site stores (ID, pwd) on server
3. When logging in, user enters pwd', server compares it to *pwd*

We have a problem

- Server stores all users' passwords
- If this file is stolen, $\mathcal{A}$ can impersonate any user

Password-Based Authentication

How to use a password:

1. User creates password *pwd* when registering for site
2. Site stores (ID, pwd) on server
3. When logging in, user enters pwd', server compares it to *pwd*

We have a problem

- Server stores all users' passwords
- If this file is stolen, $\mathcal{A}$ can impersonate any user

Can we protect passwords even if password file is stolen?

Hash The Passwords

How to use a password:

1. User creates password *pwd* when registering for site

Hash The Passwords

How to use a password:

1. User creates password *pwd* when registering for site
2. Site computes $H(pwd)$, stores $(ID, H(pwd))$ on server

Hash The Passwords

How to use a password:

1. User creates password pwd when registering for site
2. Site computes $H(pwd)$, stores $(ID, H(pwd))$ on server
3. When logging in, user enters pwd', server computes $H(pwd')$ and checks password file

Hash The Passwords

How to use a password:

1. User creates password pwd when registering for site
2. Site computes $H(pwd)$, stores $(ID, H(pwd))$ on server
3. When logging in, user enters pwd', server computes $H(pwd')$ and checks password file

Pros:

- Passwords not stored in clear
- $\mathcal{A}$ who steals password file only gets hashes of passwords

Hash The Passwords

How to use a password:

1. User creates password pwd when registering for site
2. Site computes $H(pwd)$, stores $(ID, H(pwd))$ on server
3. When logging in, user enters pwd', server computes $H(pwd')$ and checks password file

Pros:

- Passwords not stored in clear
- $\mathcal{A}$ who steals password file only gets hashes of passwords

Cons:

- $\mathcal{A}$ can build big list of hashed passwords
- When $\mathcal{A}$ gets files of hashed passwords, can see if any on the list

Hash The Passwords

How to use a password:

1. User creates password pwd when registering for site
2. Site computes $H(pwd)$, stores $(ID, H(pwd))$ on server
3. When logging in, user enters pwd', server computes $H(pwd')$ and checks password file

Pros:

- Passwords not stored in clear
- $\mathcal{A}$ who steals password file only gets hashes of passwords

Cons:

- $\mathcal{A}$ can build big list of hashed passwords
- When $\mathcal{A}$ gets files of hashed passwords, can see if any on the list
- Recovering password of any user is good enough

Hash The Passwords

How to use a password:

1. User creates password pwd when registering for site
2. Site computes $H(pwd)$, stores $(ID, H(pwd))$ on server
3. When logging in, user enters pwd', server computes $H(pwd')$ and checks password file

Pros:

- Passwords not stored in clear
- $\mathcal{A}$ who steals password file only gets hashes of passwords

Cons:

- $\mathcal{A}$ can build big list of hashed passwords
- When $\mathcal{A}$ gets files of hashed passwords, can see if any on the list
- Recovering password of any user is good enough
- Hashing is very fast – Billions of hashes / second

Add a Salt

How to use a password:

1. User creates password *pwd* when registering for site

Add a Salt

How to use a password:

1. User creates password pwd when registering for site
2. Site chooses salt $s \leftarrow \{0,1\}^n$, computes $H(s||pwd)$ and stores $(ID, s, H(s||pwd))$

Add a Salt

How to use a password:

1. User creates password pwd when registering for site
2. Site chooses salt $s \leftarrow \{0,1\}^n$, computes $H(s||pwd)$ and stores $(ID, s, H(s||pwd))$
3. When logging in, user enters pwd', server finds s, computes $H(s||pwd')$ and checks password file

Add a Salt

How to use a password:

1. User creates password pwd when registering for site
2. Site chooses salt $s \leftarrow \{0,1\}^n$, computes $H(s||pwd)$ and stores $(ID, s, H(s||pwd))$
3. When logging in, user enters pwd', server finds s, computes $H(s||pwd')$ and checks password file

Pros:

- s is long and random, can't be predicted by $\mathcal{A}$

Add a Salt

How to use a password:

1. User creates password pwd when registering for site
2. Site chooses salt $s \leftarrow \{0,1\}^n$, computes $H(s||pwd)$ and stores $(ID, s, H(s||pwd))$
3. When logging in, user enters pwd', server finds s, computes $H(s||pwd')$ and checks password file

Pros:

- s is long and random, can't be predicted by $\mathcal{A}$
- If $\mathcal{A}$ builds list of hashed passwords, it'll be useless if s is wrong

Add a Salt

How to use a password:

1. User creates password pwd when registering for site
2. Site chooses salt $s \leftarrow \{0,1\}^n$, computes $H(s||pwd)$ and stores $(ID, s, H(s||pwd))$
3. When logging in, user enters pwd', server finds s, computes $H(s||pwd')$ and checks password file

Pros:

- s is long and random, can't be predicted by $\mathcal{A}$
- If $\mathcal{A}$ builds list of hashed passwords, it'll be useless if s is wrong
- s is unique per user, so even if he constructs list after seeing s, he must repeat for each user

Add a Salt

How to use a password:

1. User creates password pwd when registering for site
2. Site chooses salt $s \leftarrow \{0,1\}^n$, computes $H(s||pwd)$ and stores $(ID, s, H(s||pwd))$
3. When logging in, user enters pwd', server finds s, computes $H(s||pwd')$ and checks password file

Pros:

- s is long and random, can't be predicted by $\mathcal{A}$
- If $\mathcal{A}$ builds list of hashed passwords, it'll be useless if s is wrong
- s is unique per user, so even if he constructs list after seeing s, he must repeat for each user

Takeaway

Always use a salt